Lecture Notes: When Models Surprise Us

Phase Transitions, Grokking, and the Test-Time Compute Revolution

LLM Lab — Executive Education Program

2026

Contents

1 Introduction: The Unifying Thread **2**

2 Topic A: Phase Transitions in Capabilities **2**
2.1 Background: Neural Scaling Laws . . . 2
2.1.1 Kaplan et al. (2020): Power-Law Scaling . . . 2
2.1.2 Hoffmann et al. (2022): The Chinchilla Scaling Law . . . 2
2.2 Emergent Abilities: Wei et al. (TMLR 2022) . . . 3
2.2.1 Scale of Emergence . . . 3
2.2.2 The Physics Analogy: Anderson's "More Is Different" (1972) . . . 3
2.3 The Mirage Rebuttal: Schaeffer et al. (NeurIPS 2023) . . . 4
2.3.1 The Core Mathematical Argument . . . 4
2.3.2 Resolution Effects . . . 4
2.3.3 The BIG-Bench Meta-Analysis . . . 5
2.3.4 Three Predictions Tested on GPT-3/InstructGPT . . . 5
2.3.5 Vision Experiments . . . 5
2.4 The Nuanced Resolution . . . 5

3 Topic B: Grokking **6**
3.1 The Discovery: Power et al. (2022) . . . 6
3.1.1 Experimental Setup . . . 6
3.1.2 Training Dynamics (Division mod 97, 50% data) . . . 6
3.1.3 Which Operations Grokked . . . 6
3.1.4 The Role of Regularisation . . . 6
3.2 Mechanistic Explanation: Nanda et al. (ICLR 2023) . . . 6
3.2.1 Setup . . . 7
3.2.2 The Learned Algorithm: Discrete Fourier Transform . . . 7
3.2.3 Quality of the Approximation . . . 7
3.2.4 Three Training Phases . . . 7
3.2.5 Progress Measures . . . 8
3.2.6 The Role of Weight Decay . . . 8
3.3 Circuit Efficiency: Varma et al. (ICLR 2025) . . . 8
3.3.1 The Efficiency Framework . . . 8
3.3.2 Three Conditions for Grokking . . . 8
3.3.3 Ungrokking . . . 9
3.3.4 Semi-Grokking . . . 9
3.4 Grokking at Scale: Li et al. (2025) . . . 9
3.4.1 Model and Setup . . . 9

3.4.2 Local Grokking . . . 9
3.4.3 Pathway Evolution Metrics . . . 9
3.5 Connection to Classical Learning Theory . . . 9
3.5.1 Violation of the Bias-Variance Tradeoff . . . 9
3.5.2 Connection to Double Descent . . . 10

4 Topic C: The Test-Time Compute Revolution **10**
4.1 The Paradigm Shift . . . 10
4.2 Foundations: Chain-of-Thought Prompting . . . 10
4.3 Test-Time Compute Scaling: Snell et al. (ICLR 2025) . . . 10
4.3.1 Formal Framework . . . 11
4.3.2 Two Strategies . . . 11
4.3.3 The Difficulty-Dependent Result . . . 11
4.3.4 The Headline Result . . . 11
4.4 DeepSeek-R1: Reasoning from Pure Reinforcement Learning . . . 12
4.4.1 Two Systems: R1-Zero and R1 . . . 12
4.4.2 GRPO: Group Relative Policy Optimisation . . . 12
4.4.3 The "Aha Moment" . . . 12
4.4.4 Benchmark Results . . . 13
4.4.5 The Readability Problem . . . 13
4.4.6 Distillation Results . . . 13
4.5 Budget Forcing: s1-32B (Muennighoff et al., 2025) . . . 13
4.5.1 The s1K Dataset . . . 13
4.5.2 Budget Forcing . . . 13
4.5.3 Results . . . 14
4.5.4 Why 1,000 Examples Suffice . . . 14
4.6 The Broader Landscape . . . 14
4.6.1 Taxonomy of Test-Time Compute Methods . . . 14
4.6.2 Self-Consistency (Wang et al., ICLR 2023) . . . 14
4.6.3 Tree of Thoughts (Yao et al., NeurIPS 2023) . . . 14
4.6.4 OpenAI o1 and o3 . . . 14
4.6.5 The System 1 / System 2 Analogy . . . 15
4.7 Two Axes of Scaling: Strategic Implications . . . 15

5 Synthesis: What the Audience Should Take Away **15**

1 Introduction: The Unifying Thread

These notes accompany the 30-minute presentation "When Models Surprise Us." They provide the technical depth behind each slide, suitable for a presenter with a master's-level background in computer science or machine learning.

The three topics share a single message: **the dynamics of learning in large language models violate our deepest intuitions about how systems acquire capabilities.**

1. **Phase transitions in capabilities:** Scaling a model can produce discontinuous jumps in what it can do — but some of these jumps may be measurement artifacts.
2. **Grokking:** A model can memorise its training data perfectly, appear to stop learning, and then — much later — suddenly generalise. This challenges the bias-variance tradeoff that has governed statistics for over a century.
3. **Test-time compute:** A smaller model that "thinks longer" at inference time can outperform a model 14× its size. This opens a second axis of scaling that fundamentally changes the economics of deploying LLMs.

2 Topic A: Phase Transitions in Capabilities

2.1 Background: Neural Scaling Laws

Before discussing emergence, we must understand the baseline expectation: *smooth, predictable scaling.*

2.1.1 Kaplan et al. (2020): Power-Law Scaling

Kaplan et al. [2] established that cross-entropy loss follows power-law relationships with model size N (non-embedding parameters), dataset size D (tokens), and compute C (FLOPs):

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad N_c = 8.8 \times 10^{13},\ \alpha_N = 0.076 \tag{1}$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad D_c = 5.4 \times 10^{13},\ \alpha_D = 0.095 \tag{2}$$

$$L(C) = \left(\frac{C_{\min}}{C}\right)^{\alpha_C}, \quad C_{\min} = 3.1 \times 10^{8},\ \alpha_C = 0.050 \tag{3}$$

These hold when each variable is scaled independently without the others becoming a bottleneck. The key implication: multiplying parameters by 10 reduces loss by a factor of $10^{-0.076} \approx 0.84$ (a 16% reduction). Loss improves smoothly and predictably.

Kaplan concluded that compute-optimal allocation should scale parameters much faster than data: $N_{\text{opt}} \propto C^{0.73}$, $D_{\text{opt}} \propto C^{0.27}$. This was later corrected by Hoffmann et al.

2.1.2 Hoffmann et al. (2022): The Chinchilla Scaling Law

Hoffmann et al. [3] proposed a parametric loss function:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \tag{4}$$

where E is the irreducible entropy of natural language. Their fitted constants:

$$E = 1.69, \quad A = 406.4, \quad \alpha = 0.34, \quad B = 410.7, \quad \beta = 0.28.$$

A replication by Besiroglu et al. [4] found somewhat different values ($E = 1.82$, $\alpha = 0.35$, $\beta = 0.37$) due to numerical issues in the original optimisation, but the qualitative conclusion held.

The critical correction of Kaplan: compute-optimal allocation should scale parameters and data *equally*:

$$N_{\text{opt}} \propto C^a, \quad D_{\text{opt}} \propto C^b, \quad a \approx b \approx 0.5. \tag{5}$$

This yields the famous "20 tokens per parameter" heuristic. Chinchilla (70B parameters, 1.4T tokens) matched the performance of Gopher (280B parameters, 300B tokens) with the same compute budget.

Remark 2.1. These scaling laws describe **loss** (cross-entropy), which improves smoothly. Emergence concerns **downstream task accuracy**, which can behave very differently. This disconnect is the crux of the emergence debate.

2.2 Emergent Abilities: Wei et al. (TMLR 2022)

Definition 2.1 (Emergent Ability [5])**.** An ability is *emergent* if it is not present in smaller models but is present in larger models. Specifically, performance remains near-random for all models below some critical scale threshold, then jumps sharply to well-above-random. This emergence "cannot be predicted by extrapolating a scaling law based on small-scale models."

Wei et al. examined eight model families: GPT-3 (350M–175B), LaMDA (40M–137B), PaLM (8B–540B), Chinchilla (7B–70B), and Gopher (7.1B–280B), among others. The primary scaling variable is **training compute** (FLOPs), though parameter count is roughly proportional.

2.2.1 Scale of Emergence

The paper catalogued emergent abilities across two major evaluation suites:

- **BIG-Bench:** 67 out of ∼210 tasks showed emergent behaviour.
- **MMLU:** 51 tasks showed emergent behaviour.
- Wei later compiled 137 emergent abilities across the literature.

Concrete thresholds (approximate):

Task	**Model family**	**Emergence threshold**
3-digit addition	GPT-3	∼13B params (2.3×10^{22} FLOPs)
Modular arithmetic	GPT-3	∼13B params
Chain-of-thought reasoning	LaMDA	∼68B params (1.3×10^{23} FLOPs)
College-level exams (MMLU)	GPT-3	∼175B params (3.1×10^{23} FLOPs)
Word in Context	PaLM	∼540B params

2.2.2 The Physics Analogy: Anderson's "More Is Different" (1972)

Wei et al. explicitly invoke the framework of P.W. Anderson's seminal essay [1]. Anderson argued that **reductionism does not imply constructionism**: knowing the fundamental laws governing a system's components does not allow you to predict the emergent properties of the system at scale.

Anderson's central mechanism is **broken symmetry**: the solutions to equations can have lower symmetry than the equations themselves. His hierarchy:

X (higher level)	Y (lower level)
Solid-state physics	Elementary particle physics
Chemistry	Many-body physics
Molecular biology	Chemistry
Cell biology	Molecular biology
Psychology	Physiology
Social sciences	Psychology

Key examples from Anderson:

- **Ammonia (NH_3):** Tunnels between two mirror-image states $\sim 3 \times 10^{10}$ times per second. For larger molecules, tunnelling times exceed the age of the universe — symmetry is effectively broken by scale alone.
- **Crystallisation:** A gas with continuous translational symmetry spontaneously forms a crystal lattice with discrete symmetry. Rigidity is an emergent property.
- **Superconductivity:** Broken gauge symmetry (BCS theory). The fundamental equations were known for 30 years before the phenomenon could be explained.

The analogy to LLMs: knowing the transformer architecture and training loss does not, in Anderson's framework, allow you to predict what capabilities emerge at what scale — just as knowing quantum mechanics does not allow you to predict superconductivity from first principles.

2.3 The Mirage Rebuttal: Schaeffer et al. (NeurIPS 2023)

Schaeffer, Miranda, and Koyejo [6] (Outstanding Paper Award) argued that emergent abilities are **not a property of the model, but of the researcher's choice of evaluation metric.**

2.3.1 The Core Mathematical Argument

Suppose per-token probability of correctness $p(N)$ improves smoothly with scale N. For a task requiring a correct output sequence of length L, the **Exact String Match** metric scores:

$$\text{Accuracy}(N) = p(N)^L \tag{6}$$

This exponential creates artificial sharpness:

Per-token accuracy p	Sequence length L	Exact-match accuracy p^L
60%	4	13.0%
80%	4	40.9%
90%	4	65.6%
98%	4	92.2%

The improvement from $p = 0.6$ to $p = 0.98$ is smooth and gradual, but the exact-match metric transforms it into an apparent phase transition (13% $\rightarrow$ 92%).

2.3.2 Resolution Effects

With a test set of size T, the smallest measurable nonzero performance is $1/T$. Any true performance below $1/T$ reads as exactly zero, creating an artificial plateau that makes the eventual rise appear more sudden.

2.3.3 The BIG-Bench Meta-Analysis

More than 92% of claimed emergent abilities in BIG-Bench appeared under just two metrics: Multiple Choice Grade and Exact String Match. Both are discontinuous. Of the 39 metrics available in BIG-Bench, emergent abilities appeared with at most 5 of them.

2.3.4 Three Predictions Tested on GPT-3/InstructGPT

Schaeffer et al. tested three predictions using arithmetic tasks (2-shot multiplication of two 2-digit integers, addition of two 4-digit integers):

1. **Changing the metric:** Switching from Exact String Match to Token Edit Distance made the sharp transition disappear — performance improved smoothly and predictably. *Confirmed.*
2. **Increasing test set size:** Improving measurement resolution smoothed out apparent emergence even under nonlinear metrics. *Confirmed.*
3. **Varying sequence length:** Under Exact String Match, accuracy fell geometrically with L (as predicted by Eq. 6). Under Token Edit Distance, the decrease was approximately quasi-linear. *Confirmed.*

2.3.5 Vision Experiments

To demonstrate that emergence is not special to language models, they *induced* apparent emergent abilities in vision models (CNNs, autoencoders, Vision Transformers) on CIFAR-100:

- Under smooth metrics (Brier Score, mean per-class recall): smooth scaling.
- Under thresholded metrics analogous to Exact String Match: apparent sharp transitions.

No vision researcher has ever claimed emergent abilities — precisely because the vision community does not use harsh all-or-nothing metrics.

2.4 The Nuanced Resolution

The truth is that **both sides are partly right**:

1. **Some emergence is metric artifact:** Schaeffer's argument is mathematically rigorous and empirically confirmed for many BIG-Bench tasks.
2. **Some emergence may be real:** Tasks with composite structure (requiring k correct sub-steps) have success rates $\sim p^k$, which creates genuine threshold behaviour even under continuous per-step metrics. Multi-step reasoning tasks may exhibit real phase transitions.
3. **The unresolved question:** Whether *all* apparent emergence can be explained by metric choice and finite test-set resolution, or whether some phenomena are genuinely discontinuous, remains actively debated. The Berti et al. (2025) survey [7] provides the most balanced current assessment.

Key message for the audience: We genuinely cannot predict what the next model will be able to do. Both the optimists and the sceptics are partly right — which makes the uncertainty irreducible and strategically important.

3 Topic B: Grokking

3.1 The Discovery: Power et al. (2022)

Power et al. [8] (OpenAI, ICLR 2022 Workshop) discovered that neural networks trained on small algorithmic datasets can exhibit **delayed generalisation**: the model memorises the training data quickly, then — orders of magnitude later in training — suddenly generalises to the test set.

3.1.1 Experimental Setup

- **Tasks:** Binary operations on $\mathbb{Z}/97\mathbb{Z}$ (modular arithmetic with prime $p = 97$): addition, subtraction, division, polynomials ($x^2 + y^2$, $x^2 + xy + y^2$, $x^3 + xy$, etc.), and composition in the symmetric group S_5.
- **Data format:** Sequences "$a \circ b = c$" where c is masked and must be predicted.
- **Train/test split:** 50% of all (a, b) pairs for training; remainder for validation. A critical threshold exists between 30–40% — near 25% of data, a 1% decrease in training fraction leads to a 40–50% increase in median time to generalisation.
- **Architecture:** 2-layer decoder-only transformer, embedding dimension 128, 4 attention heads, $\sim 4 \times 10^5$ non-embedding parameters.
- **Optimiser:** AdamW, learning rate 10^{-3}, weight decay $\lambda = 1$ (notably large), $\beta_1 = 0.9$, $\beta_2 = 0.98$.

3.1.2 Training Dynamics (Division mod 97, 50% data)

Phase	Optimisation steps	What happens
Memorisation	$< 10^3$	Training accuracy $\to$ 100%
Stall	10^3–10^5	No visible improvement in test accuracy
Generalisation	$\sim 10^6$	Test accuracy $\to$ 100%

The gap between memorisation and generalisation spans roughly **three orders of magnitude** in optimisation steps. Every practitioner in the field would have stopped training during the stall phase.

3.1.3 Which Operations Grokked

All tested operations generalised except one: $x^3 + xy^2 + y \pmod{97}$ did not generalise within the optimisation budget at any training fraction up to 95%. Symmetric operations (e.g., $x + y$, $x^2 + y^2$) required less data to grok than asymmetric ones.

3.1.4 The Role of Regularisation

Weight decay was the single most important factor — it more than halved the data fraction needed for generalisation. Full-batch gradient descent was less effective than minibatch SGD, suggesting that stochastic noise also plays a role.

3.2 Mechanistic Explanation: Nanda et al. (ICLR 2023)

Nanda et al. [9] (ICLR 2023 Oral) fully **reverse-engineered the algorithm** learned by a small transformer trained on modular addition, revealing that the network discovers **discrete Fourier analysis**.

3.2.1 Setup

- **Task:** $(a + b) \bmod 113$ (prime modulus).
- **Architecture:** 1-layer transformer, embedding dimension $d = 128$, 4 attention heads (each $d/4 = 32$ dimensions), MLP hidden layer with 512 neurons, ReLU activation.
- **Training data:** 30% of all $113 \times 113 = 12{,}769$ input pairs ($\approx$3,831 training pairs).
- **Optimiser:** AdamW, lr $= 10^{-3}$, $\lambda = 1$, trained for 40,000 epochs.

3.2.2 The Learned Algorithm: Discrete Fourier Transform

The network learns to compute $(a+b) \bmod 113$ using five key frequencies $k \in \{14, 35, 41, 42, 52\}$ with angular frequencies $\omega_k = 2\pi k/113$. The algorithm proceeds in four stages:

Stage 1 — Embedding (W_E): The embedding matrix maps each integer $a \in \{0, \ldots, 112\}$ to its Fourier components:

$$a \longmapsto \big(\sin(\omega_k a),\ \cos(\omega_k a)\big) \quad \text{for each key frequency } k. \tag{7}$$

The embedding is **sparse in the Fourier basis**: only $\sim$6 frequencies have significant norm.

Stage 2 — Composition via trigonometric identities (Attention + MLP): The network computes:

$$\cos\big(\omega_k(a+b)\big) = \cos(\omega_k a)\cos(\omega_k b) - \sin(\omega_k a)\sin(\omega_k b) \tag{8}$$

$$\sin\big(\omega_k(a+b)\big) = \sin(\omega_k a)\cos(\omega_k b) + \cos(\omega_k a)\sin(\omega_k b) \tag{9}$$

This is the standard product-to-sum identity, implemented by the attention heads and MLP.

Stage 3 — Unembedding (W_U): For each candidate output $c \in \{0, \ldots, 112\}$, the network computes:

$$\cos\big(\omega_k(a+b-c)\big) = \cos(\omega_k(a{+}b))\cos(\omega_k c) + \sin(\omega_k(a{+}b))\sin(\omega_k c) \tag{10}$$

Stage 4 — Constructive interference: The logit for the correct answer $c^* = (a+b) \bmod 113$ receives constructive interference from all five key frequencies (all cosine terms equal 1 when $a + b - c^* \equiv 0 \pmod{113}$), while incorrect answers receive destructive interference. This gives c^* the largest logit.

3.2.3 Quality of the Approximation

The neuron-logit map $W_L = W_U W_{\text{out}}$ is approximately rank 10 and can be decomposed as:

$$W_L \approx \sum_k \big[\cos(\omega_k)\,\mathbf{u}_k^\top + \sin(\omega_k)\,\mathbf{v}_k^\top\big]$$

with residual Frobenius norm under **0.55%** of $\|W_L\|_F$. The final logit approximation explains $\sim$95% of variance.

3.2.4 Three Training Phases

Phase	Epochs	Characteristics
Memorisation	0–1,400	Train/excluded loss decline; test loss high
Circuit formation	1,400–9,400	Generalising circuit grows; $\|W\|_2$ falls
Cleanup	9,400–14,000	Memorising circuit pruned; test loss drops sharply

3.2.5 Progress Measures

Nanda et al. defined continuous metrics that track the formation of the generalising circuit:
Definition 3.1 (Restricted Loss)**.** Set all Fourier logit terms to zero except (a) the constant term and (b) the 20 terms corresponding to $\cos(\omega_k(a+b))$ and $\sin(\omega_k(a+b))$ for the five key frequencies. Then measure the resulting cross-entropy loss. This measures how well the Fourier circuit alone performs.
Definition 3.2 (Excluded Loss)**.** Remove only the key frequency components from the logits on the training data; measure the resulting training loss. This measures how well the memorising circuit performs *without* help from the generalising circuit.

Additional metrics include the **Gini coefficient** of Fourier component norms (measuring sparsity) and the L_2 norm of weights (tracking weight decay pressure). Crucially, these progress measures show **continuous, monotonic improvement** throughout training — even while test accuracy remains at chance. Grokking appears sudden on the test accuracy curve but is gradual in the underlying representation.

3.2.6 The Role of Weight Decay

Weight decay creates a pressure that penalises the high-norm memorising solution and favours the more parameter-efficient generalising (Fourier) solution. Without weight decay, no grokking occurs. The memorising and generalising circuits have **different norm-to-performance trade-offs**, and weight decay selects for the more efficient one.

3.3 Circuit Efficiency: Varma et al. (ICLR 2025)

Varma et al. [10] (Google DeepMind) provided a unifying theoretical framework for grokking based on **circuit efficiency**.

3.3.1 The Efficiency Framework

The total loss decomposes into two competing terms:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2 \tag{11}$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss and λ is the weight decay coefficient.
Definition 3.3 (Circuit Efficiency)**.** The efficiency of a circuit is the **logit-to-parameter-norm ratio**: the magnitude of output logits achievable per unit of L_2 parameter norm. A circuit is more efficient if it achieves the same cross-entropy loss with lower parameter norm.

3.3.2 Three Conditions for Grokking

Grokking occurs when:

1. The generalising circuit C_{gen} generalises well while the memorising circuit C_{mem} does not.
2. C_{gen} is **more efficient** than C_{mem} (larger logits per unit parameter norm).
3. C_{gen} is **learned more slowly** than C_{mem}.

The key empirical finding: C_{mem}'s efficiency **decreases** with larger training datasets (memorisation cost grows with data), while C_{gen}'s efficiency **remains constant** (the generalising algorithm's parameter cost is fixed). This creates a critical dataset size D_{crit} where the efficiencies cross over.

3.3.3 Ungrokking

Definition 3.4 (Ungrokking)**.** A network that has already grokked (achieved perfect generalisation) **regresses to near-chance test accuracy** — a reversal of grokking.

To trigger ungrokking: take a network that has grokked on a large dataset (above D_{crit}), then retrain it on a significantly smaller dataset (below D_{crit}). Because C_{mem} is now more efficient, gradient descent with weight decay shifts the network back toward memorisation.

3.3.4 Semi-Grokking

Near D_{crit}, where both circuits have comparable efficiency, the network plateaus at **partial** test accuracy rather than completing the transition to perfect generalisation.

3.4 Grokking at Scale: Li et al. (2025)

Li et al. [11] demonstrated that grokking occurs during **real LLM pretraining** at the 7B-parameter scale, confirming that it is not merely a toy phenomenon.

3.4.1 Model and Setup

- **Model:** OLMoE-1B-7B — decoder-only transformer with Mixture-of-Experts (MoE), 7B total parameters, ∼1.3B active per forward pass. 64 routed experts per layer, top-8 selected per token.
- **Training data:** ∼5.1 trillion tokens.

3.4.2 Local Grokking

Unlike the global, synchronous grokking in toy settings, LLM pretraining exhibits **asynchronous, domain-level** grokking:

- Different domains (maths, code, commonsense) are memorised at different training steps.
- Each domain takes varying amounts of additional training to generalise.
- The training loss has converged globally, but downstream benchmark performance continues to improve.

3.4.3 Pathway Evolution Metrics

The MoE architecture provides a natural lens. Li et al. defined:

1. **Edit Distance:** Levenshtein distance between different samples' expert routing sequences. During grokking, this **declines** — pathways become more structured.
2. **Pathway Consistency:** Cosine similarity between consecutive layers' expert embeddings. This **increases** during grokking — routing becomes smoother.

These metrics showed correlation $r > 0.97$ with downstream test accuracy, while training loss showed weak, inconsistent correlations. Four domains demonstrated grokking: arithmetic reasoning, code generation, commonsense QA, and domain-specific QA.

3.5 Connection to Classical Learning Theory

3.5.1 Violation of the Bias-Variance Tradeoff

In classical statistics, a model that has overfit (high variance, low bias) is expected to *stay* overfit. Grokking directly contradicts this: the model first interpolates the training data perfectly, then

transitions to perfect generalisation under continued training. This should not happen under classical assumptions.

3.5.2 Connection to Double Descent

Double descent [12] describes a non-monotonic test error curve: classical U-shape, peak at the interpolation threshold, then improvement in the overparameterised regime. Davies [13] proposed that **grokking and double descent are instances of the same phenomenon**: inductive biases (weight decay) prefer better-generalising but slower-to-learn patterns.

Three pattern types in the unified framework:

1. **Heuristics:** Fast-learning, well-generalising (Type 1).
2. **Memorisation:** Fast-learning, poorly-generalising (Type 2).
3. **Slow generalising:** Slow-learning, well-generalising, ultimately preferred by inductive biases (Type 3).

Grokking = **epoch-wise** transition from Type 2 to Type 3 dominance.
Double descent = the same transition viewed as a function of **model capacity**.

4 Topic C: The Test-Time Compute Revolution

4.1 The Paradigm Shift

The old paradigm for improving LLM performance: **make models bigger** (more parameters, more training data, more training FLOPs). The new paradigm adds a second axis: **let models think longer at inference time.**

The fundamental insight: you can trade *inference-time compute* for *model size*, and in many cases, a smaller model that thinks longer beats a larger model that answers immediately.

4.2 Foundations: Chain-of-Thought Prompting

Wei et al. [14] (NeurIPS 2022) demonstrated that including intermediate reasoning steps in few-shot exemplars dramatically improves performance on reasoning tasks:

Model / Method	GSM8K accuracy
PaLM 540B (standard prompting)	∼18%
PaLM 540B (chain-of-thought)	∼57%
Prior SOTA (fine-tuned GPT-3 + verifier)	55%

Chain-of-thought is the simplest form of test-time compute scaling: the model generates intermediate reasoning tokens that mediate the computation, spending additional FLOPs at inference time.

Crucially, CoT is an emergent ability: it only works above ∼100B parameters. For small models (<10B), CoT **hurts** performance — the models produce fluent but logically incoherent chains of thought.

4.3 Test-Time Compute Scaling: Snell et al. (ICLR 2025)

Snell et al. [15] (ICLR 2025 Oral) provided the most rigorous framework for understanding when and how to scale test-time compute.

4.3.1 Formal Framework

Given a question q with ground-truth answer $y^*(q)$, a test-time compute budget N (number of generated tokens/samples), and hyperparameters θ controlling how that budget is spent:

$$\theta^*_{q,y^*(q)}(N) = \arg\max_{\theta} \ \mathbb{E}_{y\sim\text{Target}(\theta,N,q)}\left[\mathbf{1}\{y = y^*(q)\}\right] \tag{12}$$

The optimal strategy should be **prompt-dependent**: different questions call for different allocation strategies.

4.3.2 Two Strategies

Strategy (a): Search against a process-based verifier (PRM).

- Sample N candidate solutions from the base LM.
- Score using a Process Reward Model (PRM) that provides step-level correctness judgements.
- Sub-approaches: best-of-N weighted selection, beam search against the PRM (retain top N/M candidates per step, expand M proposals each), and lookahead search with k-step rollouts.

Strategy (b): Sequential revision.

- Generate solutions **sequentially**, each conditioned on all previous attempts.
- The model iteratively refines its answer, seeing the original question plus its own prior outputs.
- Can be combined with a verifier for selection among revisions.

4.3.3 The Difficulty-Dependent Result

Which strategy is optimal depends critically on problem difficulty:

- **Easy problems** (high base pass@1): Sequential revision is most effective. Beam search shows signs of over-optimisation.
- **Medium-difficulty:** Beam search against the PRM demonstrates consistent improvements.
- **Hard problems** (very low base pass@1): Neither strategy helps much. For these, a larger pretrained model remains preferable.

The compute-optimal adaptive strategy yields a $>$**4**$\times$ **efficiency gain** over a uniform best-of-N baseline.

4.3.4 The Headline Result

In a FLOPs-matched comparison, a small base model with optimal test-time compute scaling can **outperform a 14**$\times$ **larger model** using standard decoding — but only on problems where the smaller model has a somewhat non-trivial base success rate. On the very hardest problems, the larger pretrained model still wins.

Benchmark: MATH dataset (Hendrycks et al.) with PaLM 2-S* (Codey) as the base model.

4.4 DeepSeek-R1: Reasoning from Pure Reinforcement Learning

DeepSeek-R1 [16] (January 2025, published in Nature) demonstrated that reasoning capabilities can emerge from **pure reinforcement learning**, without any human demonstrations of step-by-step reasoning.

4.4.1 Two Systems: R1-Zero and R1

DeepSeek-R1-Zero (pure RL):

- Base model: DeepSeek-V3-Base (MoE LLM).
- No supervised fine-tuning before RL — trained directly with GRPO.
- Only rule-based reward signals: accuracy (deterministic verification of boxed answers) and format compliance.

DeepSeek-R1 (with cold start): A four-stage pipeline:

1. Cold start SFT on thousands of human-curated long-CoT examples.
2. Reasoning-oriented RL with GRPO (accuracy + language consistency rewards).
3. Rejection sampling + SFT ($\sim$600K reasoning + 200K non-reasoning samples).
4. All-scenario RL for helpfulness and harmlessness.

4.4.2 GRPO: Group Relative Policy Optimisation

GRPO eliminates the critic network entirely. For each question q, sample a group of G outputs $\{o_1, \ldots, o_G\}$ from the current policy. Compute reward r_i for each. The advantage is the **z-score normalised reward within the group**:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, \ldots, r_G\})}{\text{std}(\{r_1, \ldots, r_G\})} \tag{13}$$

The objective function is:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim P(Q),\, \{o_i\} \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^{G} \Big(\min\big(\rho_i \hat{A}_i,\ \text{clip}(\rho_i, 1 \pm \epsilon) \hat{A}_i\big) - \beta\, D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \Big) \right] \tag{14}$$

where $\rho_i = \pi_\theta(o_i|q)/\pi_{\theta_{\text{old}}}(o_i|q)$ is the importance sampling ratio and π_{ref} is the initial model. This halves the memory requirements compared to PPO by avoiding a separate critic model.

4.4.3 The "Aha Moment"

During R1-Zero training, the model **spontaneously** developed self-reflection capabilities. At an intermediate checkpoint, the model began producing text like:

> *"Wait, wait. Wait. That's an aha moment I can flag here."*

Nobody told the model to think step by step. Nobody provided examples of self-correction. Through pure trial-and-error optimisation of a reward signal, the model discovered that **pausing to reconsider improves its answers.**

The emergent behaviours include: chain-of-thought reasoning, self-verification, reflection, and dynamic strategy adaptation (switching approaches mid-solution).

4.4.4 Benchmark Results

Benchmark	R1-Zero	R1 (full)	For reference: o1
AIME 2024	71.0%	79.8%	74%
MATH-500	95.9%	97.3%	—
GPQA Diamond	50.0%	71.5%	—
Codeforces (rating)	1444	2029	—

R1-Zero improved from 15.6% to 71.0% on AIME 2024 over the course of RL training. With consensus@64 sampling, R1-Zero achieves 86.7% — matching OpenAI o1.

4.4.5 The Readability Problem

R1-Zero suffers from language mixing (Chinese/English interspersed) and poor formatting — the RL reward does not penalise these, so the model uses whatever internal representation maximises accuracy. R1's cold-start SFT and language consistency rewards fix this.

4.4.6 Distillation Results

Distilled from R1 into smaller models:

Distilled model	AIME 2024	MATH-500
Qwen-1.5B	28.9%	83.9%
Qwen-7B	55.5%	92.8%
Qwen-14B	69.7%	93.9%
Qwen-32B	72.6%	94.3%
Llama-70B	70.0%	94.5%

The 1.5B distilled model outperforms GPT-4o on maths benchmarks. Distillation from a strong reasoning teacher is remarkably effective.

4.5 Budget Forcing: s1-32B (Muennighoff et al., 2025)

Muennighoff et al. [17] demonstrated that **1,000 curated training examples** are sufficient to unlock test-time scaling, beating OpenAI o1-preview on competition maths.

4.5.1 The s1K Dataset

Starting from 59,029 samples, curated through three-stage filtering:

1. **Quality filtering** (59K → 51.5K): Remove API errors, formatting issues.
2. **Difficulty filtering** (51.5K → 24.5K): Remove problems both Qwen-7B and Qwen-32B solve correctly. Use reasoning trace length as difficulty proxy.
3. **Diversity filtering** (24.5K → 1K): Classify into 50 domains; sample uniformly across domains, favouring longer traces.

53.6% of the reasoning traces in s1K are correct — the presence of incorrect traces did not prevent effective training.

4.5.2 Budget Forcing

Two operations controlling test-time compute:

- **Enforce maximum:** Append the end-of-thinking token to force early termination.

- **Enforce minimum:** Suppress the end-of-thinking token in sampling logits, then append "Wait" to the reasoning trace. This triggers the model to re-examine its work and self-correct.

Budget forcing can be applied multiple times (2×, 4×, 6×) for progressively deeper thinking.

4.5.3 Results

Model	AIME 2024
o1-preview	44.6%
s1-32B (no budget forcing)	50.0%
s1-32B (Wait 2×)	53.3%
s1-32B (Wait 6×)	57.0%

4.5.4 Why 1,000 Examples Suffice

The authors invoke the **Superficial Alignment Hypothesis** (from the LIMA paper): the model is already exposed to vast amounts of reasoning data during pretraining. The 1,000-example SFT stage does not teach reasoning from scratch — it **activates** latent reasoning capabilities already encoded during pretraining. Training took **26 minutes on 16 NVIDIA H100 GPUs**.

4.6 The Broader Landscape

4.6.1 Taxonomy of Test-Time Compute Methods

Method	Type	Year	Key mechanism
Chain-of-Thought	Sequential (prompting)	2022	Few-shot exemplars with reasoning
Self-Consistency [18]	Parallel (sampling)	2023	Multiple paths + majority vote
Tree-of-Thoughts [19]	Tree search	2023	BFS/DFS over thought candidates
OpenAI o1/o3	Sequential (learned)	2024	RL-trained internal reasoning tokens
DeepSeek-R1	Sequential (learned)	2025	GRPO-trained extended CoT
Snell et al.	Hybrid (adaptive)	2025	PRM-guided beam search + revision
s1-32B	Sequential (forced)	2025	"Wait" token injection

4.6.2 Self-Consistency (Wang et al., ICLR 2023)

Sample multiple diverse reasoning paths, extract the final answer from each, select by majority vote. Improvements over CoT: GSM8K +17.9%, SVAMP +11.0%, AQuA +12.2%.

4.6.3 Tree of Thoughts (Yao et al., NeurIPS 2023)

Organise reasoning into a tree where the LM generates multiple candidates per step, evaluates them, and uses BFS/DFS to search. GPT-4 on Game of 24: 4% with CoT → 74% with ToT.

4.6.4 OpenAI o1 and o3

OpenAI o1 (September 2024): RL-trained to produce internal reasoning tokens (hidden from user). AIME 2024: 74% pass@1.

OpenAI o3 (December 2024): Further scaling of test-time compute. AIME 2024: **96.7%**. ARC-AGI: **87.5%** (high-compute; o1 scored only 32%).

4.6.5 The System 1 / System 2 Analogy

The connection to Kahneman's dual-process theory:

- **System 1** (fast, intuitive): Standard LLM inference — a single forward pass, pattern matching, associative.
- **System 2** (slow, deliberate): Test-time compute scaling — extended reasoning, search, verification, self-correction.

The fundamental insight: without retraining the underlying LLM, you can improve its reasoning capabilities simply by allowing it to explore more possibilities. This decouples **capability acquisition** (training) from **capability deployment** (inference), creating a new strategic variable for organisations.

4.7 Two Axes of Scaling: Strategic Implications

The capability frontier now has two dimensions: train-time compute (one-time capital expense) and test-time compute (per-query operating expense). The strategic question: where on this frontier should you invest?

Key considerations:

- **High-volume, low-latency tasks** (autocomplete, classification): Invest in a fast, large model. Test-time compute overhead is too costly at scale.
- **High-stakes, low-volume tasks** (complex analysis, planning, coding): Invest in a smaller model with extended thinking. The per-query cost is acceptable when accuracy matters.
- **Difficulty matters:** Test-time compute is most effective for medium-difficulty problems. For very easy problems, it is wasteful. For very hard problems, it does not help — a larger model is needed.

5 Synthesis: What the Audience Should Take Away

1. **Unpredictability of capabilities:** Phase transitions mean we cannot reliably forecast what the next model will do. Strategic planning must account for this uncertainty.
2. **Delayed understanding:** Grokking shows that models can develop capabilities long after training loss has converged. For safety-critical deployments, post-training evaluation is not a one-time event.
3. **A new cost structure:** Test-time compute creates a new trade-off axis. The right deployment strategy depends on task difficulty, volume, latency tolerance, and accuracy requirements.
4. **The honest answer to "do we understand these systems?":** Partially. We can reverse-engineer small models (Nanda's Fourier circuits). We can empirically characterise scaling behaviour (Chinchilla, Snell). But we cannot predict emergence, and we do not fully understand why grokking happens at scale. The frontier of understanding is advancing, but it lags behind the frontier of capability.

References

[1] P.W. Anderson. More Is Different. *Science*, 177(4047):393–396, 1972.

[2] J. Kaplan, S. McCandlish, T. Henighan, T.B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling Laws for Neural Language Models. *arXiv:2001.08361*, 2020.

[3] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, et al. Training Compute-Optimal Large Language Models. *NeurIPS*, 2022.

[4] T. Besiroglu, E. Erdil, M. Barnett, and J. You. Chinchilla Scaling: A Replication Attempt. *arXiv:2404.10102*, 2024.

[5] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, et al. Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research (TMLR)*, 2022.

[6] R. Schaeffer, B. Miranda, and S. Koyejo. Are Emergent Abilities of Large Language Models a Mirage? *NeurIPS*, 2023. Outstanding Paper Award.

[7] L. Berti, F. Giorgi, and G. Kasneci. Emergent Abilities in Large Language Models: A Survey. *arXiv:2503.05788*, 2025.

[8] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. *ICLR Workshop*, 2022.

[9] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress Measures for Grokking via Mechanistic Interpretability. *ICLR*, 2023. Oral.

[10] V. Varma, R. Shah, Z. Kenton, J. Kramar, and R. Kumar. Explaining Grokking Through Circuit Efficiency. *ICLR*, 2025.

[11] Z. Li, C. Fan, and T. Zhou. Where to Find Grokking in LLM Pretraining? *arXiv:2506.21551*, 2025.

[12] M. Belkin, D. Hsu, S. Ma, and S. Mandal. Reconciling Modern Machine Learning Practice and the Bias-Variance Trade-off. *PNAS*, 116(32):15849–15854, 2019.

[13] X. Davies, L. Langosco, and D. Krueger. Unifying Grokking and Double Descent. *arXiv:2303.06173*, 2023.

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q.V. Le, and D. Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS*, 2022.

[15] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. *ICLR*, 2025. Oral.

[16] DeepSeek-AI, D. Guo, D. Yang, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*, 2025. Published in *Nature*.

[17] N. Muennighoff, Z. Yang, W. Shi, X.L. Li, L. Fei-Fei, et al. s1: Simple Test-Time Scaling. *arXiv:2501.19393*, 2025.

[18] X. Wang, J. Wei, D. Schuurmans, Q.V. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*, 2023.

[19] S. Yao, D. Yu, J. Zhao, I. Shafran, T.L. Griffiths, Y. Cao, and K. Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS*, 2023.